(a) (1,0 pontos) Determine a função de tempo $T(n)$, separando explicitamente o custo da parte iterativa e o custo das chamadas recursivas, e escreva a relação de recorrência completa associada ao algoritmo.

(b) (2,0 pontos) Resolva a recorrência obtida no item anterior utilizando o **método da árvore de recursão**, apresentando: o custo total de cada nível, o número de níveis (profundidade) e o custo total do algoritmo a partir desses dois valores. Em seguida, classifique a complexidade assintótica final utilizando a notação mais apropriada (O, Ω ou Θ), justificando a escolha da notação com base no resultado obtido.

(Questão Extra – ENADE 2021) (0,5 pontos)

Existe um grande número de implementações para algoritmos de ordenação. Um dos fatores a serem considerados, por exemplo, é o número máximo e médio de comparações que são necessárias para ordenar um vetor com n elementos. Diz-se também que um algoritmo de ordenação é estável se ele preserva a ordem de elementos que são iguais. Isto é, se tais elementos aparecem na sequência ordenada na mesma ordem em que estão na sequência inicial. Analise o algoritmo abaixo, onde a é um vetor e "i, j, lo e hi" são índices do vetor:

```
algoritmo ordena(A, lo, hi)
    se lo < hi então
        p := particao(A, lo, hi)
        ordena(A, lo, p - 1)
        ordena(A, p + 1, hi)

algoritmo particao(A, lo, hi)
    pivot := A[hi]
    i := lo
    repita para j := lo até hi
        se A[j] < pivot entao
        troca A[i] com A[j]
        i := i + 1
    troca A[i] com A[hi]
    return i
```

Com relação ao algoritmo apresentado, avalie as afirmações a seguir e as julguem em verdadeiro ou falso.

(a) O algoritmo precisa de um espaço adicional O(n) para a pilha de recursão.

(b) O algoritmo apresentado é um algoritmo de ordenação recursivo e estável.

(c) O algoritmo precisa, em média, de O(n log n) comparações para ordenar n itens.

(d) O uso do primeiro elemento do vetor como "pivot" é mais eficiente que usar o último.

É correto apenas o que se afirma em:

- A e C.
- B e D.
- C e D.
- A, B e C.
- A, B e D.

Que seus acertos sejam críticos e seus erros sejam nulos. Que a Força esteja com você durante cada questão!

(g) (F) Um algoritmo recursivo é sempre menos eficiente, em termos de complexidade assintótica de tempo, do que sua versão iterativa equivalente, já que toda recursão implica reprocessamento de subproblemas.

(h) (F) No Teorema Mestre, quando $n^{\log_b a}$ domina assintoticamente $f(n)$ (Caso 1), o resultado correto é $T(n) = \Theta(f(n))$.

(i) (V) Ao resolver uma recorrência da forma $T(n) = aT(n/b) + f(n)$ pelo Teorema Mestre, quando $f(n)$ cresce assintoticamente mais rápido que $n^{\log_b a}$ (Caso 3), a condição de regularidade $a \cdot f(n/b) \leq c \cdot f(n)$, para algum $c < 1$, precisa ser verificada antes de concluir que $T(n) = \Theta(f(n))$.

(j) (F) Um algoritmo com dois laços aninhados dependentes de n, mas cujo laço interno executa sempre um número **constante** de iterações (independente de n), ainda assim possui complexidade $O(n^2)$.

2. (2 pontos) Um sistema de e-commerce precisa verificar, no momento do fechamento do pedido, se algum item do carrinho de compras está indisponível em estoque. O sistema percorre um vetor de n itens do carrinho e, para cada item, consulta se ele está disponível. Assim que encontra o **primeiro** item indisponível, o algoritmo interrompe a busca e retorna sua posição; caso nenhum item indisponível seja encontrado, retorna -1.

Linha / Instrução	Custo	Vezes
`VerificaEstoque(Carrinho, n):`		
`1. para i = 0 até n - 1:`		
`2.  se Carrinho[i].disponivel == falso:`		
`3.  retorne i`		
`4. retorne -1`		

Considerando o **pior caso** (nenhum item do carrinho está indisponível, ou o item indisponível está na última posição do vetor):

(a) Descubra a expressão de $T(n)$ baseado na construção do pseudocódigo acima. (1,0 ponto)

(b) Simplifique $T(n)$ e indique a complexidade assintótica Θ deste algoritmo, justificando. (1,0 ponto)

3. (3,0 pontos) Um sistema industrial realiza a análise automática de sensores utilizando o algoritmo abaixo:

```
function analiseSensores(A, n){
    if(n <= 1)
        return 1;
    contador = 0;
    for(i = 0; i < n; i++){
        contador++;
    }
    return analiseSensores(A, n/2) + analiseSensores(A, n/2) + contador;
}
```

Com base no algoritmo apresentado, responda:

Universidade de Fortaleza

	AV1	AV2	AV3
1ª ch			
2ª ch			

Curso: Ciência da computação Disciplina: Cana Código/Turma: T198-06

Professor/a: Caio Marques Data: 14/09/26

Aluno/a: Luis Guilherme Barreto Lisboa Matrícula: 2517344

5,4

Recomendações e regras para a avaliação

- A *prova* vale de 0 a 7 (+0,5 extra) e tempo de duração de 100 minutos;
- *É* **terminantemente proibida qualquer interação** com outros estudantes durante a realização da prova. O descumprimento desta regra implicará **atribuição de nota ZERO (0,0)**.
- Não é permitido utilizar **qualquer dispositivo eletrônico** para obtenção de informações durante a prova, tais como celulares, tablets, smartwatches ou computadores e também não é permitido consultas a qualquer material de apoio que não seja entregue pelo professor.
- O aluno deve desenvolver a questão com a finalidade de mostrar o resultado final. Questões com respostas corretas, porém sem o devido desenvolvimento, **NÃO serão consideradas**;
- Esta prova contém 3 questões.

---

1. (2,0 pontos) Julgue os itens a seguir como **Verdadeiro (V)** ou **Falso (F)**, marcando o parêntese correspondente e justificando **brevemente** cada julgamento (0,2 ponto por item, sendo 0,1 pela marcação e 0,1 pela justificativa correta):

   (a) (V) A notação $O(n^2)$ representa um limite **superior** para o crescimento do tempo de execução, ou seja, garante apenas que o algoritmo não é pior que n^2, mas não impede que ele também seja, por exemplo, $O(n)$.

   (b) (V) A notação $\Omega(n)$ representa um limite **inferior** de tempo de execução, garantindo que o algoritmo nunca executará em tempo menor que n, para qualquer entrada de tamanho n suficientemente grande.

   (c) (F) A notação $\Theta(n \log n)$ só pode ser aplicada quando as funções que limitam o algoritmo superior e inferiormente (O e Ω) coincidem na mesma taxa de crescimento, a menos de constantes multiplicativas.

   (d) (V) Ao somar as complexidades de dois trechos independentes de um algoritmo, um $O(n)$ e outro $O(n^2)$, a complexidade total do algoritmo é $O(n^2)$, pois a notação assintótica considera apenas o termo de maior ordem de crescimento.

   (e) (F) Em um algoritmo recursivo, a ausência de um caso base não compromete sua corretude, pois a pilha de chamadas eventualmente se esgota e o programa retorna um resultado válido.

   (f) (V) Um algoritmo puramente iterativo (sem qualquer chamada recursiva) sempre utiliza espaço adicional $O(1)$ de memória, independentemente das estruturas de dados criadas ou preenchidas dentro do laço.